NEA

Regex practice site

Table of Contents

Analysis:....4
Problem:....4
Research:....4
Research of the end user:....5
Requirements:....6
Documented design:....6
Preface:....6
Client-Server model:....6
Hierarchy diagrams:....7
System flowcharts:....8
Object-oriented design:....12
Database design:....12
Algorithms:....15
Data structures:....16
HCI (Human-computer interaction) / Front end design:....17
Technical solution:....17
Git:....17
Libraries:....18
Source tree:....19
Testing:....22
Signup router....23
Login router....24
Logout router....25
Create router....25
Search router....26
Attempt submission router....27
Appendix:....27
src/server.ts....27
src/db/db.ts....30
src/routes/account.route.ts....32
src/routes/create.route.ts....32
src/routes/delete.route.ts....34
src/routes/exception.route.ts....35
src/routes/index.route.ts....35
src/routes/leaderboard.route.ts....36
src/routes/login.route.ts....37
src/routes/logout.route.ts....38
src/routes/redirect.route.ts....39

src/routes/search.route.ts.....40
src/routes/signup.route.ts.....44
src/routes/solution.route.ts.....46
src/routes/solve.route.ts.....47
src/routes/submit_attempt.route.ts.....48
src/routes/user.route.ts.....54
src/client/create.ts.....57
src/client/solve.ts.....60

Analysis:

Problem:

Despite regex being the most efficient and cleanest tool for pattern matching and text manipulation, there are little resources to improve being able to solve regex problems. Being experienced in reading technical documentation and indulging in programming resources, regular expressions are surprisingly ignored. However, the issue lies beyond learning the basic theory of regex, and instead in being able to piece together a solution that arises from an array of requirements.

The end user, Patrick, is passionate about software that abstracts the complexity from the user above such as compilers/interpreters and parsers. The problem is that once the abstract overview of these programs is understood, the data processing modules are heavily undocumented since with regular expressions, they can be simply read from the source code rather than explained. After finding this out through my own knowledge and primary research, and trying my best to learn about the topic, it was obvious that the skill curve was too large, it would lead to one of two consequences. A regex guide would be utilised each time you had to solve a problem with regex, or you'd have to learn it whole, also dedicating time to create the resources and data needed for practice.

I asked whether he would like me to make him a regex practice program, and he agreed.

Research:

For programming itself, the best resources are the most practical oriented since programming is a skillet that you cannot attain with pure knowledge or memorisation.

The closest computational topic to regular expressions are algorithms, which both share similarities, but the biggest is that regular expressions are more specific. As a result, I though researching ways people learn algorithms for inspiration. The most popular website for learning algorithms – and job-interview problems – is Leetcode. The site is characterised by giving you instructions, the ability to use different programming languages, and testing your code against a multitude of test cases. The site encourages "grinding" out the problems; putting as much effort into improvement as possible in order to improve problem solving skills. However, the most appealing part of the site is the test case nature since it can be utilised in my case.

Research of the end user:

- Q. Have you ever avoided using regular expressions in your code in favour of less efficient methods because writing a pattern was too difficult?
- A. “I hate to admit it, but yes, quite a lot. I usually find an alternative method in the standard library of the language I’m using.”
- Q. How often do you encounter regular expressions, and how confident do you feel interpreting them?
- A. “I have very little confidence for interpreting regular expressions, and this makes me feel 10x worse considering that they’re everywhere...”
- Q. When you have to write a regular expression, how do you do it? Do you read documentation or use a website like regex101?
- A. “I usually use stack overflow if I’m attempting to do something relatively common, but if not that, I end up using AI.”
- Q. Do you feel that existing resources properly prepare you to solve regex problems practically, or do they not step beyond theory?
- A. “I can commend documentation (especially language docs) when it comes to regex since they give you the tools to use regex, but besides that it’s a disproportionate amount of theory content against practical resources.”
- Q. On a website like leetcode, what do you find most motivating: is it seeing your rank, progress or simply passing test cases?
- A. “Mostly passing the test cases – which is extremely satisfying – but rank is definitely up there too. Seeing my history isn’t as motivating though.”
- Q. Would you prefer problems to be ordered by difficulty or just freely browsable?
- A. “I can’t decide, being able to order and sort problems in a number of ways usually helps when trying to find something specific, but freely browsing is also enjoyable in a way, it lets me make sure I haven’t left anything out.”
- Q. How important is it to you to see progress of your improvement, for example through a rating or history of previously solved problems?
- A. “Fairly important. I’m not super competitive when it comes to ratings, but it’s a lot better to be aware of my level, I guess it’s important for my improvement in that regard.”
- Q. Would you like to be able to create new problems yourself, or just solve existing ones?
- A. “Definitely create my own problems, it would allow me to learn extra about regular expressions, revise them, and share with others.”

Requirements:

- Allow practising regex,
- Give immediate feedback upon solving a problem against test data on whether it passes or fails,
- The system must include a rating system,
- Allow authenticated users to create create and submit their own problems,
- The system must support searching and filtering capabilities, both for deepening knowledge and allowing to share with others,
- Signup/login must be implemented,
- Exceptions should be handled well,
- The system must validate submitted regex server side before storing them, ensuring that the pattern is valid.

Documented design:

Preface:

The first objective for the project was to set up the base layers. Just like the concept of abstraction layers, I believed that the same approach would suit the development of this project. Starting off, my biggest worry was struggling to find all the information I need for my project, and potentially having to stretch out development. Going into this, I'm aware that the web has a lot of small details you have to be aware of. To counter this, I decided that figuring out how to set up the frame/skeleton should take precedence.

Client-Server model:

Since the client requested a website, I'll be working with a client-server model, and more specifically HTTP – the protocol of the web. The most important concept in HTTP is the request and response paradigm: requests are sent by the client and a response is given back by the server, allowing two way communication.

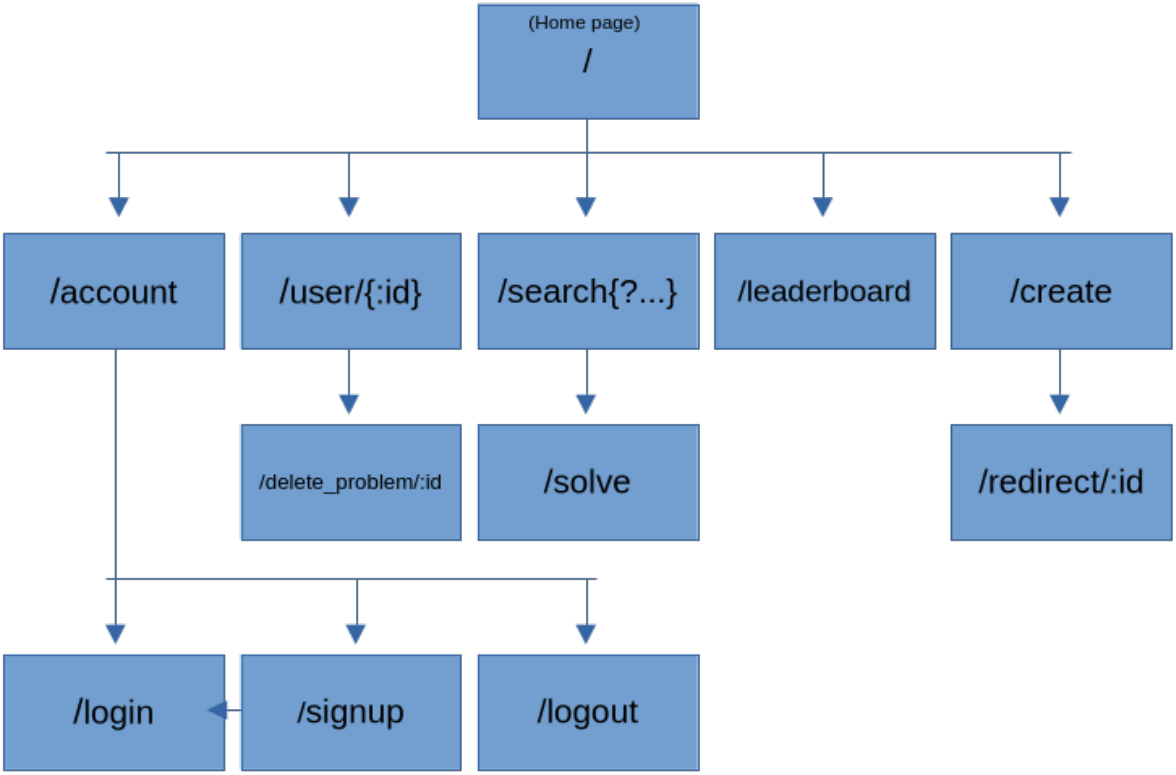
They are multiple types of requests, such as GET, POST, DELETE etc.

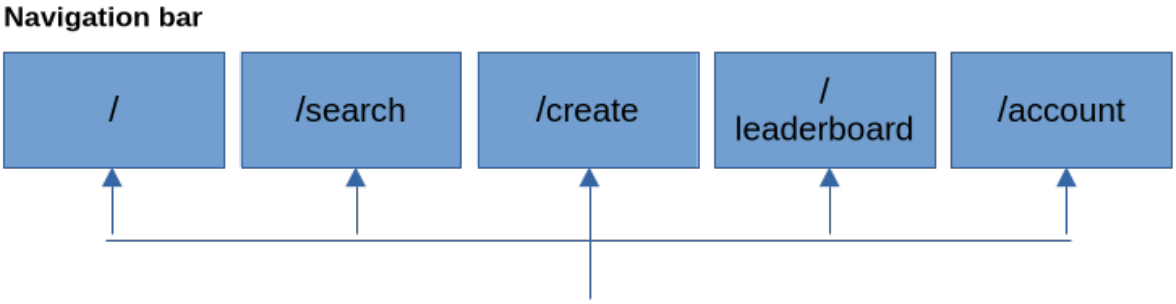
Most of my code is going to revolve around HTTP routes; code that exists to manage incoming requests.

My client and server code is also going to be vastly different. The goal of the client side code is to provide interactivity and to display data. The server side is where most processing is going to occur, and the client side code is going to reflect it, ensuring that the server won’t respond with a client error code. The client side is going to revolve around fetch API and the DOM. Fetch API is a JavaScript feature that allows HTTP requests to easily be made, and there isn’t a single piece of client side program that doesn’t include it. The DOM (or Document Object Model) is another API that connects web pages to scripts, this has also been used to change the appearance of the frontend when required.

Hierarchy diagrams:

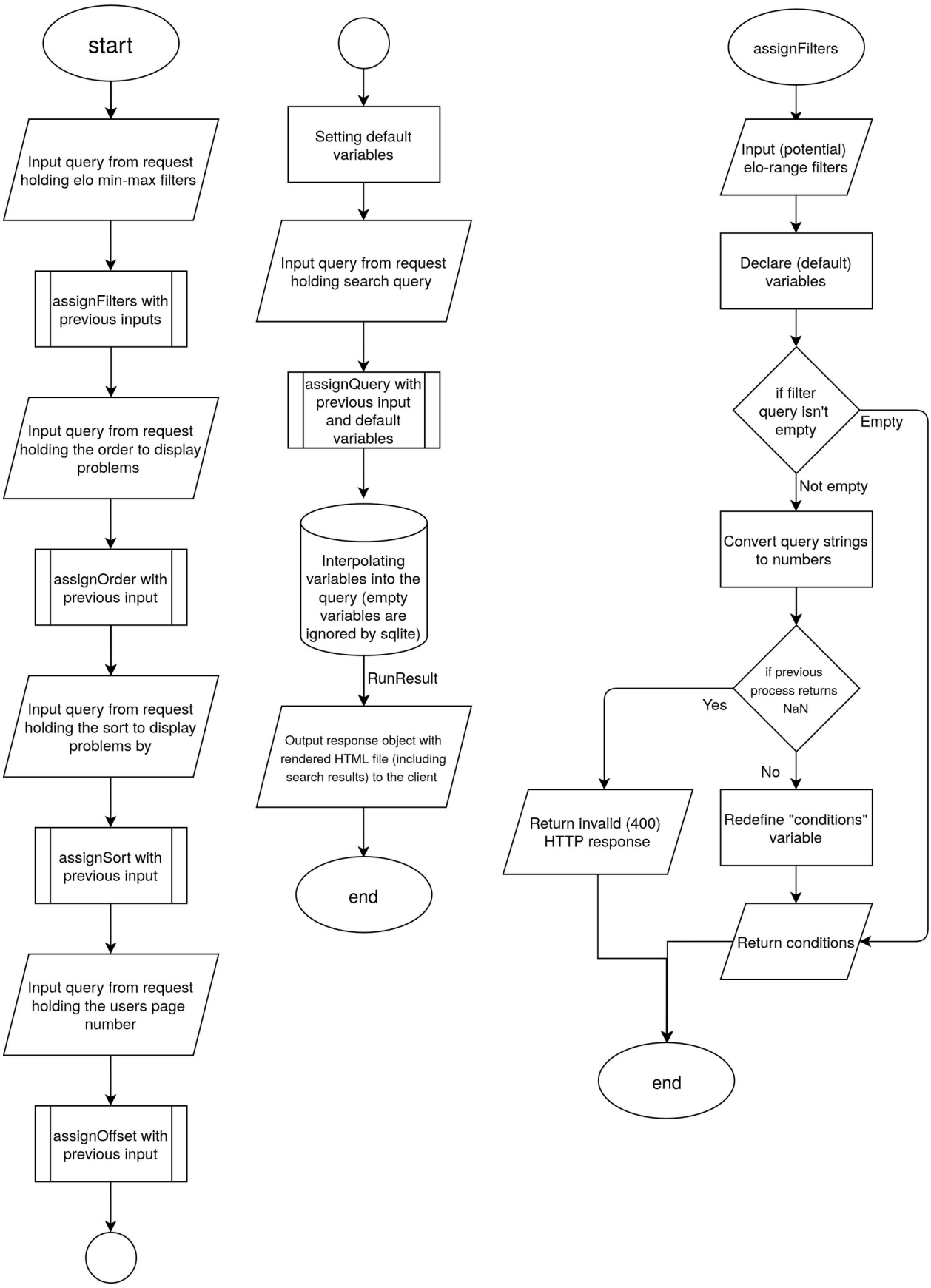
The hierarchy below displays all GET endpoints, with levels assigned by navigability (hence why the navigation bar is included).

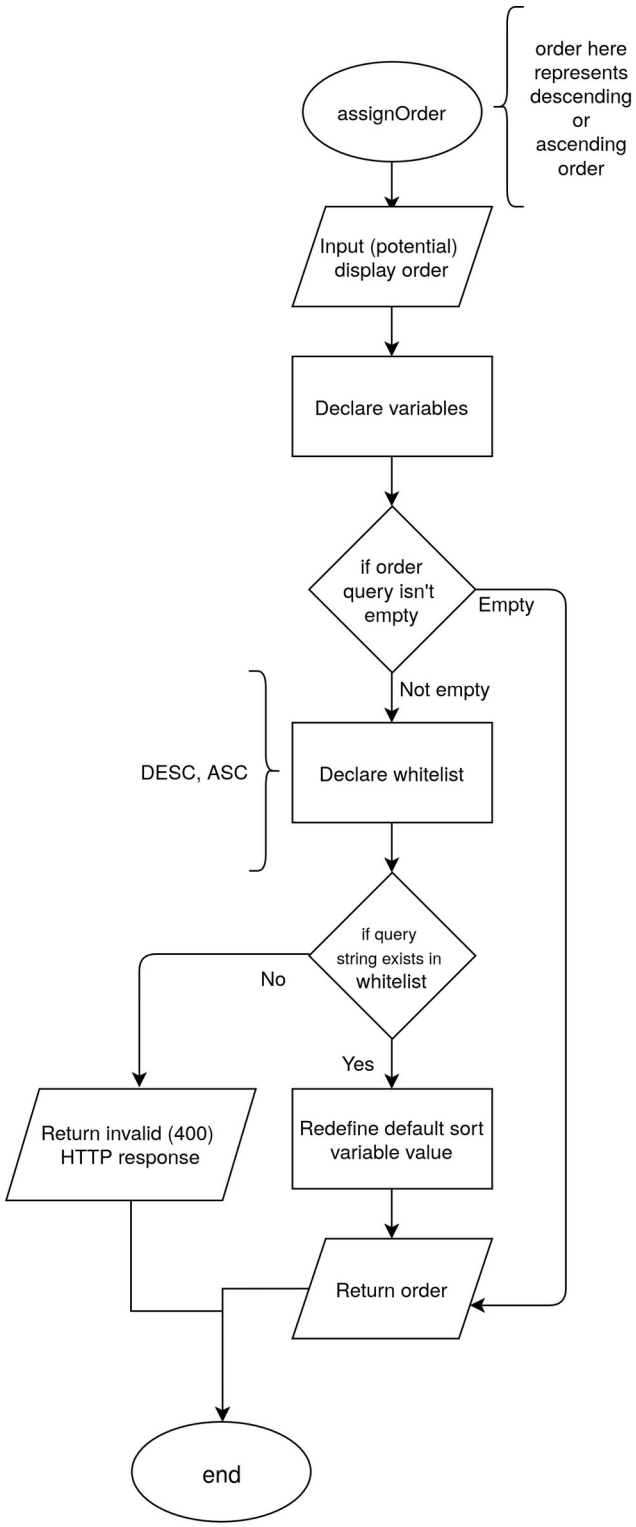


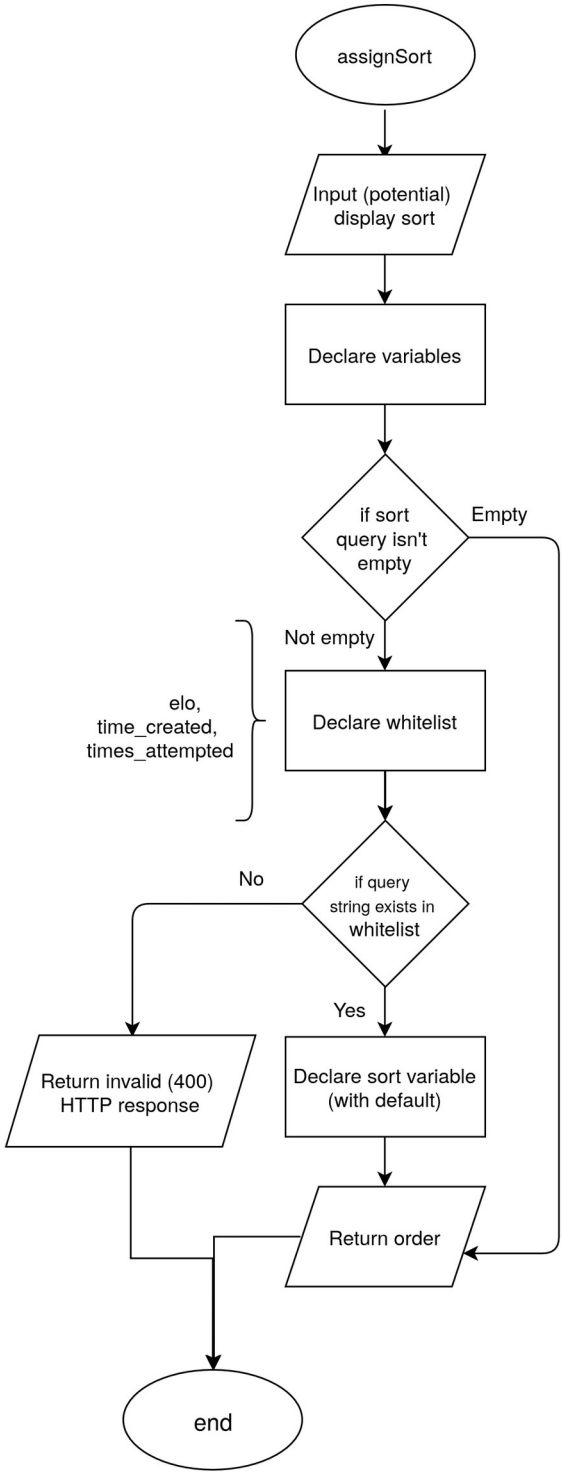


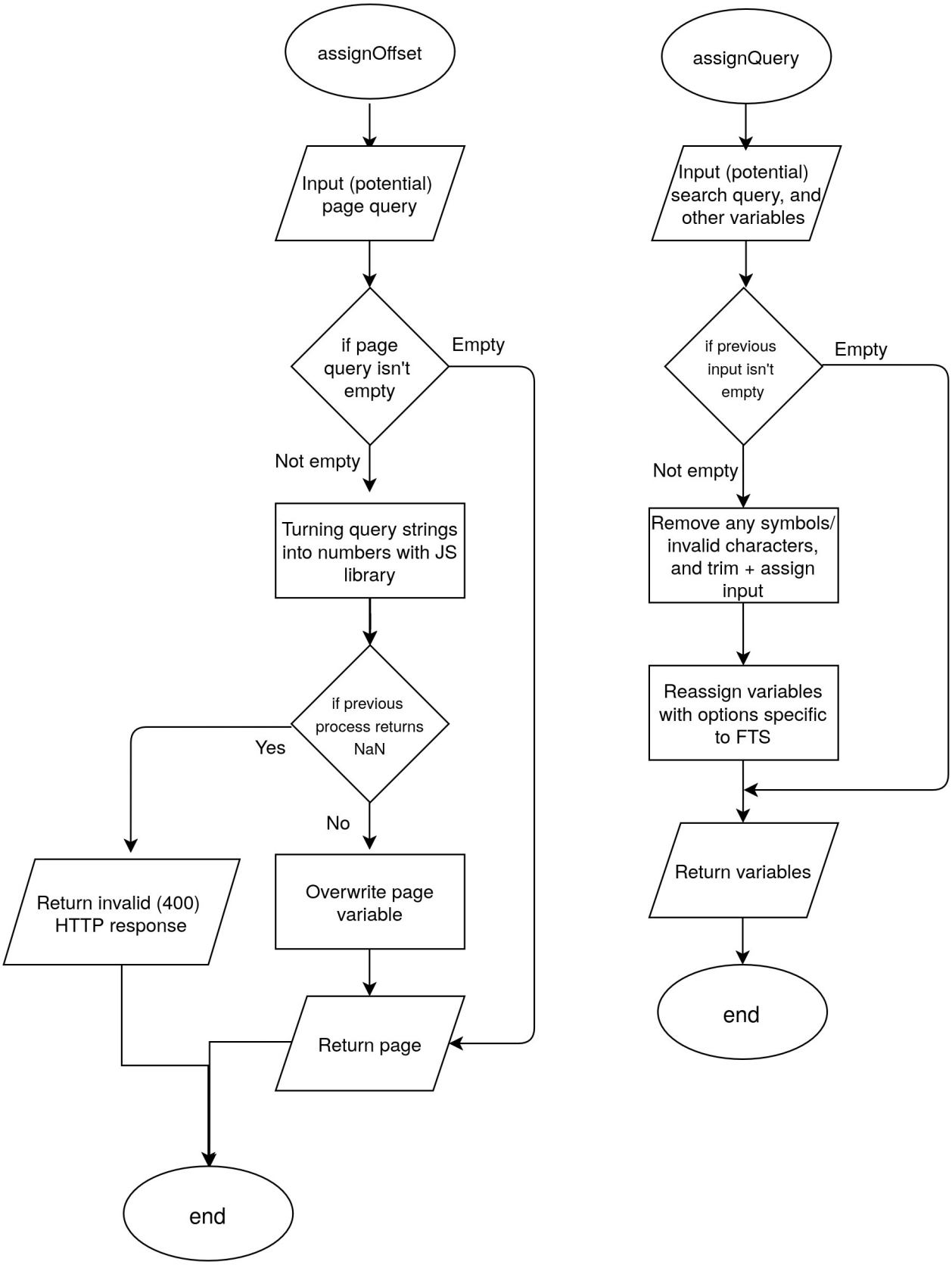
System flowcharts:

The flowcharts below show the design for the search router. Some of the detail has been abstracted, and some conditions in practice represent multiple.


start
Input query from request holding elo min-max filters
assignFilters with previous inputs
Input query from request holding the order to display problems
assignOrder with previous input
Input query from request holding the sort to display problems by
assignSort with previous input
Input query from request holding the users page number
assignOffset with previous input
Setting default variables
Input query from request holding search query
assignQuery with previous input and default variables
Interpolating variables into the query (empty variables are ignored by sqlite)
RunResult
Output response object with rendered HTML file (including search results) to the client
end
assignFilters
Input (potential) elo-range filters
Declare (default) variables
if filter query isn't empty
Empty
Not empty
Convert query strings to numbers
if previous process returns NaN
Yes
No
Return invalid (400) HTTP response
Redefine "conditions" variable
Return conditions
end


assignOrder
order here represents descending or ascending order
Input (potential) display order
Declare variables
if order query isn't empty
Empty
Not empty
DESC, ASC
Declare whitelist
if query string exists in whitelist
No
Yes
Return invalid (400) HTTP response
Redefine default sort variable value
Return order
end


assignSort
Input (potential) display sort
Declare variables
if sort query isn't empty
Empty
Not empty
elo, time_created, times_attempted
Declare whitelist
if query string exists in whitelist
No
Yes
Return invalid (400) HTTP response
Declare sort variable (with default)
Return order
end


assignOffset
Input (potential) page query
if page query isn't empty
Empty
Not empty
Turning query strings into numbers with JS library
if previous process returns NaN
Yes
No
Return invalid (400) HTTP response
Overwrite page variable
Return page
end
assignQuery
Input (potential) search query, and other variables
if previous input isn't empty
Empty
Not empty
Remove any symbols/ invalid characters, and trim + assign input
Reassign variables with options specific to FTS
Return variables
end

Object-oriented design:

The project is mostly based around an imperative approach/paradigm because I couldn't find any practical use of complex object-oriented design. But object-oriented features have been used throughout.

The whole project works around the idea that each code module computes it's own complex job with input before passing the results to another side of the model (client/server/database). An advantage of this approach is that each module simply doesn't manage much data on its own, this ensures that data flow occurs in a structured and meaningful manner. However, this makes it difficult to have enough relationships between data/code at any one location for a complex OOP model to be beneficial.

I recognise that If I was to do complex DOM tasks (on the client side), then complex OOP would suit it more than a functional paradigm, especially if I was to (create|edit|display|remove) a more-than-small amount of HTML elements.

I've been able to incorporate some simple classes across my project - especially for my elo algorithm – with some benefits including encapsulation, composition and abstraction of constant/intermediary values used in the algorithm.

Despite that, my programming is widespread with composition, most functions being made up of even smaller functions with clear relationships to the super-function. Interfaces are essential to my code, more specifically to the fetching of objects (e.g. from database) that contain values within. By doing so, the code can show what is only necessary by hiding details regarding types (the "type" of variable that is used: number, string etc.).

Database design:

The database is the key component of the extended client server model I'll be implementing (DB ↔ Server ↔ Client), so it's critical that I get it right. For starters, the data I'll have to store is: User information, Problems, Attempts. This isn't data to warrant multiple databases, so those three will be tables within one general purpose database.

I decided to use SQLite3 over any other relational database system since it's server-less, and so I don't have to set up more networking besides the HTTP server. Having a database server wouldn't provide any benefit for this project, and the "better-sqlite3" library fixes any problem that would have made it less viable.

A simple usage of SQL is:

```
SELECT * FROM Problems
WHERE creator = (@username)
ORDER BY time_created DESC;
```

The snippet above gathers all the data on the same row from the Problems table where a creator is equal to variable (@username), ordered by column time_created in descending order.

I also need to implement a search function, but there is one big limitation: SQL's "WHERE … LIKE '…';" is precise. This poses multiple steep issues. First of all, if you misspell or incorrectly capitalise the search, you won't get receive the expected problem. Using the LIKE operator will also lead to efficiency problems, since it will look through every row to find the pattern. Efficiency isn't the biggest issue with the stock method, but it does mean that the project won't be able to scale up if required.

So to implement a search function, Full Text Search (FTS) will be utilised, which is included with SQLite. It includes fast text matching, a reversed index and the ability to rank by relevance/rank. The biggest trade off comes in the form of complexity and setup, with the only relevant sources of information being Wikipedia (for the abstract) and the SQLite documentation. To set this up, a virtual FTS table has to be created:

```
CREATE VIRTUAL TABLE IF NOT EXISTS Problems_fts USING fts5(
	title,
	creator,
	content='Problems', -- Points to the table it's based on,
	content_rowid='problem_id' -- and the primary key of that table.
);
```

The below triggers are self explanatory, they will conditionally execute additional queries whenever insertions/deletions to the Problems table have been made to mimic the it.

```
CREATE TRIGGER IF NOT EXISTS fts_insert AFTER INSERT ON Problems BEGIN
	INSERT INTO Problems_fts(rowid, title, creator)
```

```
	VALUES (new.problem_id, new.title, new.creator);
END;
```

The deletion method below marks the contents unavailable rather than traditionally deleting them.

```
CREATE TRIGGER IF NOT EXISTS fts_delete AFTER DELETE ON Problems BEGIN
	INSERT INTO Problems_fts(Problems_fts, rowid, title, creator)
	VALUES ('delete', old.problem_id, old.title, old.creator);
END;
```

This is the recommended method, however, it will require rebuilding the FTS table if storage becomes a problem:

```
INSERT INTO Problems_fts(Problems_fts) VALUES('rebuild');
```

This can simply be ran using the SQLite CLI program alongside updating the system during server downtime/maintenance.

Practical usage of FTS is:

```
let results: sqlite3.RunResult | unknown[] = db.prepare(`
	SELECT ${columns}
	FROM Problems
	${innerJoin}
	WHERE ${conditions}
	ORDER BY ${sort}
	LIMIT (@limit) OFFSET (@offset);
`).all({
	query: query,
	limit: LIMIT,
	offset: OFFSET
```

```
});
```

The combined SQL and Typescript code above uses interpolated strings to flexibly create a query during runtime, also reusing code and avoiding unnecessary conditional branching. A key detail is the innerJoin string, which is needed for FTS search, however, if the user didn't input a search string, the variable won't contain any characters ("") and will be ignored by SQLite.

Aggregate SQL functions were also used in a few places. The first query is used to check whether a Problem with a specific problem_id exists in the database:

```
SELECT COUNT(1)
AS exists
FROM Problems
WHERE problem_id = (@id);
```

The "AS" command has been used so that the returned object would be "DATA.exists" rather than "DATA.COUNT(1)". Even thought the value of the inner object variable wouldn't change, exists equals 1 (true) is a lot more readable, and thus suitable for use. Another use was to calculate the average elo of problems a user has solved.

```
SELECT AVG(Problems.elo)
AS avg_elo
FROM Attempts
INNER JOIN Problems ON Attempts.problem_id = Problems.problem_id
WHERE Attempts.username = ?
```

Algorithms:

The key algorithm for this project is going to be the elo algorithm. The elo rating system is a method of calculating the relative skill of players. It's commonly used in zero-sum games (where somebody's win is another's loss) such as sports, competitive video games, and

most notably chess. This is going to be used to determine both the difficulty of user created problems as well as user skill. A rating system will also allow users to search for problems around their skill level. I'll be using the standard Elo system over the Glicko, otherwise I'd be over engineering a problem that requires a more simple solution.

The theory of the elo algorithm is as follows:

1. The current ratings of the user and problem are fetched from the database.
2. The estimated ability (probability of winning) is calculated using the below formula where: *Ea is the estimated ability of player A, and Ra + Rb are ratings for player A + B.*

$$Ea = \frac{1}{1 + 10^{(Rb - Ra)/400)}}$$

3. Calculate new ratings using two separate formulas (since the previous could be generalisable to either player).

$$Ra^1 = Ra + K\ x\left(Sa - Ea\right)$$

$$Rb^1 = Rb + K\ x\left(1 - Sa - Eb\right)$$

4. Remove (truncate) any decimals from the elo calculation results.
5. Return and insert values into the database.

I use multiple algorithms in my search router to both parse and validate user input, before deciding what values should be interpolated into the database search query. It works by setting default values, that are either left alone or overwritten with valid user input. User values are parsed from the request object, which also includes a query object (created using URLSearchParams). The high level overview of the algorithms are displayed by the previous flowcharts. These algorithms do one of 3 distinct techniques: String to number conversion (also checking for NaN), comparing string against whitelist (using arrays to ensure user input fits server side constraints) or regex cleaning (getting rid of all symbols or unwanted characters before the SQL query).

Data structures:

I used a variety of data structures, but with the focus of only using those where necessary:

- (JavaScript) Objects: Variables with multiple variables or functions. All SQLite SELECT statements return an object in JavaScript. Objects are usually interfaced.

- Arrays: These were used for validation, more specifically comparing values against. Lists weren't used since arrays have a fixed size set during runtime.
- JSON: Although not a data structure, it's sent between the client and server as a means of communication. Two most notable uses are for: sending data to the server, and sending error messages to the client.

HCI (Human-computer interaction) / Front end design:

Since I am creating a website, the frontend has to be adequate for repetitive use. For this to be achieved I need to consider the following:

- Intractability
- Quantity of information displayed
- Navigation (the user shouldn't have to use the URL bar)
- Presentation of information
    - Conditional presentation/interactivity

To fulfil these aims I will be using EJS and BootstrapJS. Embedded JavaScript (EJS) is a templating language that will allow me to change HTML with JavaScript, as well as render the HTML on the server side with appended data before sending the it to the user. This saves time by not having repeated client side code appending data onto the HTML when unnecessary.

BootstrapJS is a library that implements many frontend elements like buttons or inputs, and it's the second most used JavaScript library (w3techs 2026). I won't need to use much of it, but it's going to prove useful for quickly spinning parts of the HCI at no cost.

Technical solution:

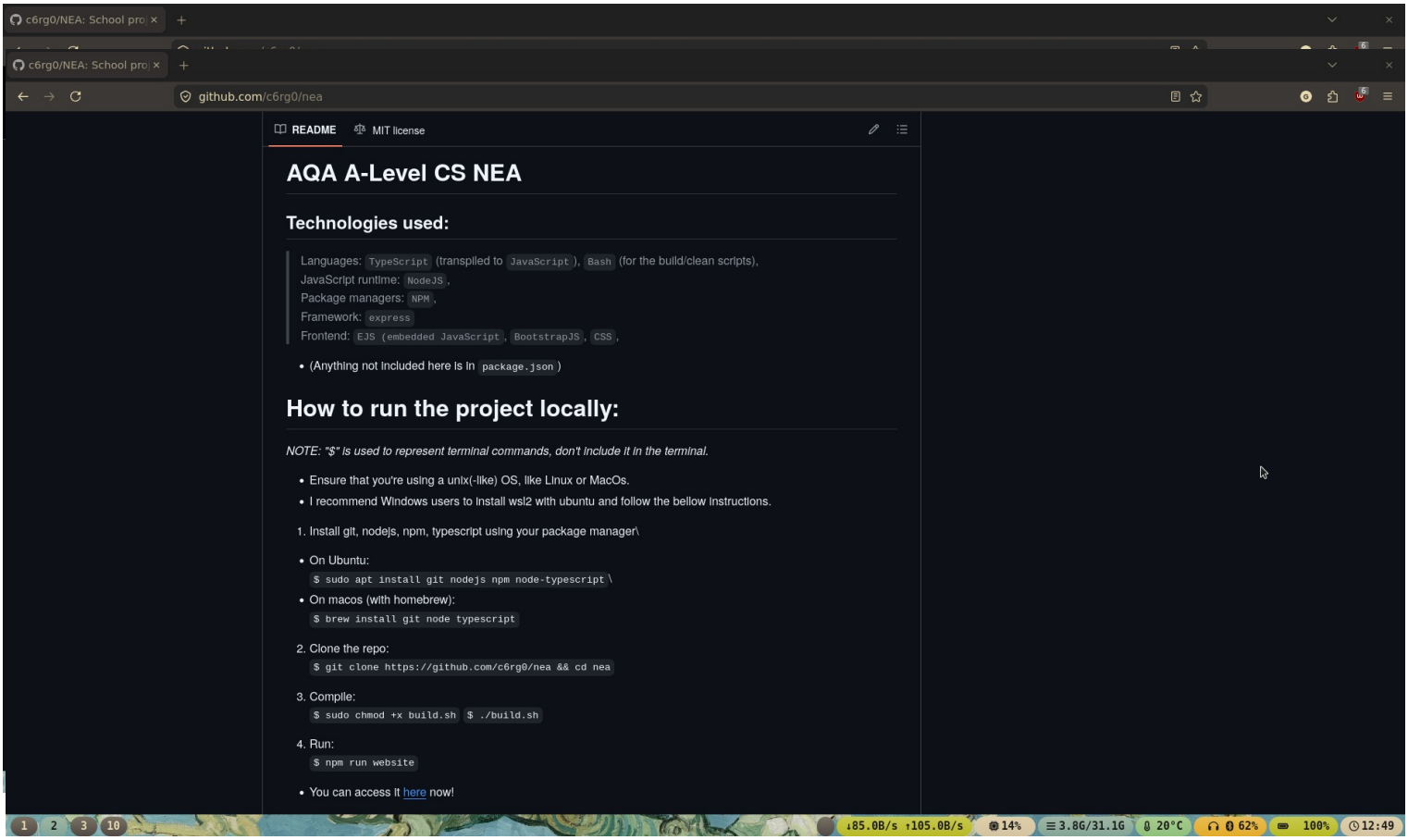
Git:

Git is an open source version control tool originally created for the development of the Linux kernel (one of the most sophisticated software projects) that tracks differences in code between snapshots (formally called commits). I learnt to use git through this project, and it serves multiple benefits.

The most important reason is how it doubles as a safety net. I can freely edit my code without setting any changes in stone. If I want the current directory to reflect a previous commit, all I have use is `git reflog` and `git reset –soft (commit hash)` in the terminal. It's also easier to find new bugs with `git show (commit hash)`, and using the output – which is based on the `diff` command – to find what change caused it.

The tool also serves as documentation, since you can browse through all changes that you have made.

In this project, git was additionally paired with GitHub. I did so because GitHub lets me backup my source code (and other small-ish files) for free. There is a size limit of 500mb for a private repository, so I decided to make it public since any images/videos might weigh more than expected. I additionally included an MIT license so anyone can use my code, without a license it's under CC, which shouldn't be used for code.



Libraries:

A lot of libraries were utilised in this project, but generally, most have been used for 1-2 features. Before I list them, I used the Node Package Manager (NPM). A package manager is a program that simplifies software installation (especially in the form of dependency management) and management (e.g. deletion).

An important note is that the bellow isn't the complete curation of libraries used. A lot are dependencies and NodeJS modules (the standard library), even if they haven't been used. I included a docs/SBOM.json file generated by GitHub for every single package used.

Library packages to run the website:

typescript: The language used in the project, package (library) for transpiling,

express: The web framework used, used for HTTP routing and helpers (e.g. redirecting),

body-parser: Allows parsing HTTP request bodies (in the name),

better-sqlite3: JavaScript implementation of SQLite3,

ejs: Embedded JavaScript, explained in the HCI section of documented design,

bcrypt: Password hashing function meant to protect against rainbow table/brute force attacks with the use of salts,

express-session: For allowing users to stay logged while they're on the site.

Development packages; pre-made types for some of the packages above:

@types/express

@types/better-sqlite3

@types/bcrypt

@types/express-session

Source tree:

("$" specifies a terminal command)

```
$ tree
.
├── LICENSE
├── README.md
├── build.sh
├── clean.sh
├── dist
│   ├── db
│   │   ├── db.db
│   │   ├── db.db-shm
│   │   └── db.db-wal
│   ├── public
│   │   ├── css
│   │   │   └── style.css
```

```
│   │   ├── fonts
│   │   │   ├── Ubuntu-Light.woff2
│   │   │   ├── Ubuntu-Medium.woff2
│   │   │   ├── Ubuntu-Regular.woff2
│   │   │   └── Ubuntu-font-license.txt
│   │   └── js
│   ├── routes
│   └── views
│       ├── account.ejs
│       ├── create.ejs
│       ├── create_success.ejs
│       ├── delete.ejs
│       ├── index.ejs
│       ├── leaderboard.ejs
│       ├── login.ejs
│       ├── search.ejs
│       ├── signup.ejs
│       ├── solve.ejs
│       └── user.ejs
├── docs
│   ├── ...
├── package-lock.json
├── package.json
├── src
│   ├── client
│   │   ├── create.ts
│   │   ├── delete.ts
│   │   ├── login.ts
│   │   ├── logout.ts
│   │   ├── search.ts
│   │   ├── signup.ts
```

```
│   │   └── solve.ts
│   ├── db
│   │   └── db.ts
│   ├── routes
│   │   ├── account.route.ts
│   │   ├── submit_attempt.route.ts
│   │   ├── create.route.ts
│   │   ├── delete.route.ts
│   │   ├── exception.route.ts
│   │   ├── index.route.ts
│   │   ├── leaderboard.route.ts
│   │   ├── login.route.ts
│   │   ├── logout.route.ts
│   │   ├── redirect.route.ts
│   │   ├── search.route.ts
│   │   ├── signup.route.ts
│   │   ├── solution.route.ts
│   │   ├── solve.route.ts
│   │   └── user.route.ts
│   └── server.ts
├── tests
├── tsconfig.client.json
└── tsconfig.json

19 directories, 68 files
```

14 directories, 62 files

Note that before I ran the tree command, all compiled code (including the node_modules directory) was removed with a bash script. The reason being is that those files aren't representative of the work I have done.

Dist is short for “distribution”, and it’s the directory from where you would start the website. It contains compiled output, frontend files (.ejs + .css) and the SQLite database. I’ve split the project directory like this to ensure that compiled output remain distinct from the source code.

The tsconfig* files are compiler configurations. They specify compiler input/output, project root, and whether certain compiler/code features should be enabled or not.

Package.json is a configuration file made by the package manager, it includes what packages you want to install and other miscellaneous information (like project creator, version number, or description). The package lock file enumerates all dependencies and the version installed, this improves reproducibility.

Testing:

For my project, the most efficient and reproducible method of testing will involve a networking tool called CURL. It’ll allow me to send HTTP requests from the terminal, and it’s going to be used to represent client side code. The reason being is that it’s significantly easier to represent the frontend with a single command, anyone can use this tool, so it’s essential that I also run server side validation.

Another thing worth mentioning is the use of HTTP status codes. Since the project uses HTTP, status codes are given by the server and represented by the client side (e.g. the client side will display an error if the server gives an error). A list brief summary:

- 1xx: Informational responses (not used)
- 2xx: Successful responses
- 3xx: Redirection messages
- 4xx: Client error responses
- 5xx: Server error messages

The MDN web docs has all status codes documented.

How CURL is being used:

- “curl” is the command,
- “-s” is for silent mode (just shows what I specifically want),
- “-o /dev/null” writes output and discards it using /dev/null (purely displaying the output),
- “-w %{http_code}” outputs the HTTP status code,

- “-X … ” specifies request type,
- “-H … ” specifies HTTP header type,
- “-d … ” specifies data to append, usually JSON or URLSearchParams,
- “-c” for specifying and creating a cookie,
- “-b” for specifying and using a cookie.
- “\” for allowing the command span multiple lines (rather than stay a singular line)

Signup router

Test number	***Test type***	***Test command***	***Expected outcome***	***Actual outcome***	***Pass/ Fail***
1	Valid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{"username": "TEST_USER", "password": "Password123!"}'`	200	200	Pass
2	Invalid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{"username": "TEST_USER", "password": "Password123!"}'`	409	409	Pass
3	Invalid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{"username": "TEST_USER_2", "password": "Password123"}'`	412	412	Pass
4	Erroneous	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{}'`	412	412	Pass
5	Boundary	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{"username": "TEST_USER_123456", "password": "Password123!345678901234"}'`	200	200	Pass
6	Boundary	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{"username": "T", "password": "aB345!"}'`	200	200	Pass

7	Exceptional	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{"username": "TEST_USER_1234567", "password": "Password123!3456789012345"}'`	412	412	Pass
8	Exceptional	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/signup \ -H "Content-Type: application/json" \ -d '{"username": "", "password": "aB34!"}'`	412	412	Pass

Test 1 is sending a correct HTTP POST request with a username longer than 1 character, and a password that is: longer than 6 characters, including at least both upper/lower case, a symbol and number. 2. This is ran straight after test 1, and it's to see what will happen if a user attempts to create a user with a duplicate name. 3. Testing whether the database will accept an invalid password: without at least one upper/lower case letter, number or symbol, even if over 6 characters long. 4. No data is sent to the database. 5. Testing the upper boundary: max username length of 16 and max password length of 24 (with upper/lower, symbol number included). 6. Lower boundary: minimum username length of 1, and minimum password length of 7 (with upper/lower, symbol number included). 8. Tests what occurs above the bound: username length 17, password length 25 (upper/lower, symbol number included)

Login router

Test number	***Test type***	***Test command***	***Expected outcome***	***Actual outcome***	***Pass/ Fail***
8	Valid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/login \ -H "Content-Type: application/json" \ -d '{"username": "TEST_USER", "password": "Password123!"}' \ -c cookies.txt`	200	200	Pass
9	Invalid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/login \ -H "Content-Type: application/json" \ -d '{"username": "TEST_USER", "password": "Pasword123!"}'`	401	401	Pass
10	Erroneous	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/login \ -H "Content-Type: application/json" \`	412	412	Pass

		`-d '{}'`			

8. Shows a valid login (user input adhering to the same rules as the signup). 9. Demonstrating the result of using an incorrect password. 10. Attempting to login with no data.

Logout router

Test number	***Test type***	***Test command***	***Expected outcome***	***Actual outcome***	***Pass/ Fail***
11	Valid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/logout`	200	200	Pass

11. Demonstrating the logout router, using it without logging in before won't throw an error, but the frontend button to logout is only displayed if logged in.

Create router

Test number	***Test type***	***Test command***	***Expected outcome***	***Actual outcome***	***Pass/ Fail***
12	Valid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/create \ -H "Content-Type: application/json" \ -d '{"title": "TEST", "instruction": "TEST", "answer": "^[a-z]+$", "test_data": "hello"}' \ -b cookies.txt`	200	200	Pass
13	Invalid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/create \ -H "Content-Type: application/json" \ -d '{"title": "TEST", "instruction": "TEST", "answer": "^[a-z]+$", "test_data": "hello"}'`	401	401	Pass
14	Erroneous	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/create \ -H "Content-Type: application/json" \ -d '{"title": "TEST", "instruction": "TEST", "answer": "^[0-9]+$", "test_data": "hello"}' \ -b cookies.txt`	412	412	Pass
15	Erroneous	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/create \`	412	412	Pass

		`-H "Content-Type: application/json" \`
		`-d '{"title": "TEST", "instruction": "TEST",`
		`"answer": "[invalid", "test_data": "hello"}' \`
		`-b cookies.txt`			

12. Valid problem creation, user defined regex passed test, includes the correct data and a cookie (for authentication, the browser does this automatically). 13. Valid user defined regex, but without authentication (on the frontend you can't submit the creation form without authentication, but the server should be able to handle it too). 14. Incorrect user defined regex ("hello" includes no numbers 0-9). 15. Incorrect use of a character class, supposed to fail test and crash the router without exception handling (a response means it was able to handle it).

Search router

Unlike the other routes, this uses a defaulting system (described in the algorithm section of documented design). This means that incorrect URLSearchParam queries will be ignored, however, incorrect values within valid URLSearchParam queries will send a 4xx error.

Test number	***Test type***	***Test command***	***Expected outcome***	***Actual outcome***	***Pass/ Fail***
16	Valid	`curl -s -o /dev/null -w "%{http_code}" -X GET "http://localhost:8000/search"`	200	200	Pass
17	Valid	`curl -s -o /dev/null -w "%{http_code}" -X GET "http://localhost:8000/search?order=ASC&fl=100&fu=1000&sort=time_created&q=title+more_title"`	200	200	Pass
18	Invalid	`curl -s -o /dev/null -w "%{http_code}" -X GET "http://localhost:8000/search?order=ACC&fl=hundred&fu=thousand&sort=timeCreated&q=$*&$*"`	412	412	Pass
19	Erroneous	`curl -s -o /dev/null -w "%{http_code}" -X GET "http://localhost:8000/search?random=random"`	200	200	Pass

16. Tests a regular GET request, router will use default options for querying the database. 17. Tests correct URL search parameters with correct values. 18. Correct URL search parameters that contain wrong values. 19. Incorrect URL search parameters, this is simply going to return to defaults.

Attempt submission router

Test number	*Test type*	*Test command*	*Expected outcome*	*Actual outcome*	*Pass/ Fail*
20	Valid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/submit_attempt \ -H "Content-Type: application/json" \ -d '{"correct": "false", "problemId": "1"}'`	200	200	Pass
21	Valid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/submit_attempt \ -H "Content-Type: application/json" \ -d '{"correct": "false", "problemId": "1"}' \ -b cookies.txt`	200	200	Pass
22	Invalid	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/submit_attempt \ -H "Content-Type: application/json" \ -d '{"correct": "false", "problemId": "17"}'`	404	404	Pass
23	Erroneous	`curl -s -o /dev/null -w "%{http_code}" -X POST http://localhost:8000/submit_attempt \ -H "Content-Type: application/json" \ -d '{"correct": "false", "problemId": "one"}'`	404	404	Pass

20. Sends a correct attempt submission without authentication, database only increments “times_solved” in the Problem table. 21. Valid correct attempt submission, this time with authentication (alongside 200, this also inserted a new attempt into the Attempt table, and updated user/problem elo). 22. Sending a request for a problem that doesn’t exist. 23. Same as 23, but using a wrong format, still returns 404 since it still doesn’t exist.

Appendix:

NOTE: Not all of my code is appended, for use https://github.com/c6rg0/nea to view all code.

src/server.ts

```
import express from "express";
const APP = express();
APP.set("view engine", "ejs");
```

```
import bodyParser from "body-parser";
APP.use(express.json());
APP.use(bodyParser.urlencoded({extended: true}));

import path from "path";
APP.use(express.static(path.join(__dirname, "public")));

import session from "express-session";
declare module "express-session" {
      interface SessionData {
            user: string,
      }
}

const SESSION_MIDDLEWARE = session({
      // (with coreutils, in the terminal):
      // $ date | sha256sum
      secret:
"41db0fe8c13891866f2f92ecda4e587e000391aa8b9de40952808a3e19f796c6",
      resave: false,
      saveUninitialized: false,
});

let cookiePerm: boolean = true;
if (cookiePerm == true) {
      APP.use(SESSION_MIDDLEWARE);
}

import { DBSetup } from "./db/db";
DBSetup();

import sqlite3 from "better-sqlite3";
const DB = sqlite3("./db/db.db", { verbose: console.log });

import { indexRouter } from "./routes/index.route";
import { signupRouter} from "./routes/signup.route";
import { loginRouter } from "./routes/login.route";
import { createRouter } from "./routes/create.route";
import { redirectRouter } from "./routes/redirect.route";
import { searchRouter } from "./routes/search.route";
import { solveRouter } from "./routes/solve.route";
import { solutionRouter } from "./routes/solution.route";
import { attemptRouter } from "./routes/submit_attempt.route";
import { accountRouter } from "./routes/account.route";
import { logoutRouter } from "./routes/logout.route";
import { userRouter } from "./routes/user.route";
```

```
import { leaderboardRouter } from "./routes/leaderboard.route";
import { deleteRouter } from "./routes/delete.route";
import { elseRouter } from "./routes/exception.route";

const INDEX = indexRouter();
const ACCOUNT = accountRouter();
const ATTEMPT = attemptRouter(DB);
const CREATE = createRouter(DB);
const LEADERBOARD = leaderboardRouter(DB);
const LOGIN = loginRouter(DB);
const LOGOUT = logoutRouter();
const REDIRECT = redirectRouter(DB);
const SEARCH = searchRouter(DB);
const SIGNUP = signupRouter(DB);
const SOLVE = solveRouter(DB);
const SOLUTION = solutionRouter(DB);
const USER = userRouter(DB);
const DELETE = deleteRouter(DB);
const ELSE = elseRouter();

APP.use("/", INDEX);
APP.use("/signup/", SIGNUP);
APP.use("/login/", LOGIN);
APP.use("/create/", CREATE);
APP.use("/redirect/", REDIRECT);
APP.use("/search/", SEARCH);
APP.use("/solve/", SOLVE);
APP.use("/solution/", SOLUTION);
APP.use("/submit_attempt/", ATTEMPT);
APP.use("/account/", ACCOUNT);
APP.use("/logout/", LOGOUT);
APP.use("/user/", USER);
APP.use("/leaderboard/", LEADERBOARD);
APP.use("/delete_problem/", DELETE);
APP.use(ELSE);

const PORT = 8000;
const URL = "http://localhost:" + PORT;
try{
      APP.listen(PORT, () => {
            console.log("Website running on: " + URL);
            console.log("(Press [Ctrl + c] to end the process) \n");
      });
} catch (error){
      console.log("Critical networking error: \n");
      console.log(error + "\n");
```

```
	console.log("Exiting process.");
	process.exit(1);
}
```

src/db/db.ts

```
import sqlite3 from "better-sqlite3";

export function DBSetup(){
	try {
		const DB = new sqlite3("db/db.db", { verbose: console.log });
		DB.pragma("journal_mode = WAL \n");
		DB.exec(`
			CREATE TABLE IF NOT EXISTS Users(
				-- The primary key is string, helps reduce the number of
SQL queries required
				username TEXT PRIMARY KEY UNIQUE NOT NULL,
				password TEXT NOT NULL,
				elo INTEGER DEFAULT 400,
				time_created INTEGER DEFAULT (strftime('%s', 'now'))
				-- time is in milliseconds since the UNIX epoch
(1/1/1970); UNIX time
			);

			CREATE TABLE IF NOT EXISTS Problems(
				problem_id INTEGER PRIMARY KEY AUTOINCREMENT,
				title TEXT NOT NULL,
				creator TEXT NOT NULL,
				instruction TEXT NOT NULL,
				test_data TEXT NOT NULL, -- Test case a users solution
is used against
				answer TEXT NOT NULL,
				elo INTEGER DEFAULT 400,
				times_attempted INTEGER DEFAULT 0,
				times_solved INTEGER DEFAULT 0,
				time_created INTEGER DEFAULT (strftime('%s', 'now'))
			);

			-- https://www.sqlite.org/fts5.html
			CREATE VIRTUAL TABLE IF NOT EXISTS Problems_fts USING fts5(
				title,
				creator,
				content='Problems', -- Points to the table it's based
on,
```

```
                        content_rowid='problem_id'
                  );

                  CREATE TRIGGER IF NOT EXISTS fts_insert AFTER INSERT ON
Problems BEGIN
                        INSERT INTO Problems_fts(rowid, title, creator)
                        VALUES (new.problem_id, new.title, new.creator);
                  END;

                  CREATE TRIGGER IF NOT EXISTS fts_delete AFTER DELETE ON
Problems BEGIN
                        INSERT INTO Problems_fts(Problems_fts, rowid, title,
creator)
                        VALUES ('delete', old.problem_id, old.title,
old.creator);
                  END;

                  CREATE TABLE IF NOT EXISTS Attempts(
                        attempt_id INTEGER PRIMARY KEY AUTOINCREMENT,
                        username TEXT NOT NULL,
                        problem_id INTEGER NOT NULL,

                        -- 0 = false, 1 = true
                        solved INTEGER DEFAULT 0,

                        tries INTEGER DEFAULT 1,
                        time_submitted INTEGER DEFAULT (strftime('%s', 'now')),
                  FOREIGN KEY (username)
                        REFERENCES Users(username)
                  FOREIGN KEY (problem_id)
                        REFERENCES Problems(problem_id)
                        -- Removes attempts on problems that were deleted
                        ON DELETE CASCADE
                  );
            `);

            return;

      } catch (error) {
            console.log("Critical db schema error: \n");
            console.log(error + "\n");
            console.log("Exiting process.");
            process.exit(1);
            // 1 is a C-style failure code
      }
}
```

src/routes/account.route.ts

```
import { Request, Response, Router } from 'express'

export function accountRouter(){
	const ROUTER = Router();

	ROUTER.get("/", (req: Request, res: Response) => {
		if (!req.session.user){
			return res.render("account", { login: false });
		}
		res.render("account", { login_status: req.session.user, login:
true});
	});

	ROUTER.all("/", (req: Request, res: Response) => {
		res.set("Allow", "GET");
		res.status(405).json({ error: "HTTP nethod not allowed" });
	});

	return ROUTER;
}
```

src/routes/create.route.ts

```
import { Request, Response, Router } from "express";
import sqlite3 from "better-sqlite3";

export function createRouter(DB: sqlite3.Database){
	const ROUTER = Router();

	ROUTER.get("/", (req: Request, res: Response) => {
		if (req.session.user){
			res.render("create", { login: true } );
			return;
		}

		res.render("create", { login: false } );
			return;
		});

	ROUTER.post("/", (req: Request, res: Response) => {
		const USER_INPUT = req.body;
		if (req.session.user) {
```

```
                if (!USER_INPUT.title || !USER_INPUT.instruction || !
USER_INPUT.test_data || !USER_INPUT.answer){
                        return res.status(412).json(
                                { error: "Missing required values" }
                        );
                }


                // testing regex
                try {
                        const ANSWER_REGEX = new RegExp(USER_INPUT.answer);
                        if (!ANSWER_REGEX.test(USER_INPUT.test_data)){
                                return res.status(412).json({ error: "Invalid
regex" });
                        }
                } catch (error){
                        return res.status(412).json({ error: error });
                }

                const PROBLEM_INSERT = DB.prepare(`
                        INSERT INTO Problems
                        (title, creator, instruction, test_data, answer)
                        VALUES
                        (@title, @creator, @instruction, @test_data, @answer);
                `);

                PROBLEM_INSERT.run({
                        title: USER_INPUT.title,
                        creator: req.session.user,
                        Instruction: USER_INPUT.instruction,
                        test_data: USER_INPUT.test_data,
                        answer: USER_INPUT.answer
                });

                return res.status(200).send();
        }

        return res.status(401).json({ error: "Please login first" });
    });

    ROUTER.all("/", (req: Request, res: Response) => {
        res.set("Allow", "GET, POST");
        res.status(405).json({ error: "HTTP nethod not allowed" });
    });

    return ROUTER;
```

```
}
```

src/routes/delete.route.ts

```
import { Request, Response, Router } from 'express'
import sqlite3 from "better-sqlite3";

export function deleteRouter(DB: sqlite3.Database){
	const ROUTER = Router();

	interface problemType {
		creator: string
	}

	ROUTER.get("/:id", (req: Request, res: Response) => {
		const ID = req.params.id;
		const USER = req.session.user;

		const PROBLEM = DB.prepare(`
			SELECT *
			FROM Problems
			WHERE problem_id = (@id);
		`).get({ id: ID }) as problemType;

		if (PROBLEM){
			if (USER === PROBLEM.creator){
				res.render("delete", { authorized: true, problem:
PROBLEM });
			} else {
				res.render("delete", { authorized: false, problem: {}
});
			}
		} else {
			res.render("delete", { authorized: false, problem: {} });
		}
	});

	ROUTER.delete("/:id", async (req: Request, res: Response) => {
		const ID = req.params.id;
		const USER = req.session.user;

		const PROBLEM = DB.prepare(`
			SELECT creator
			FROM Problems
			WHERE problem_id = (@id);
```

```
            `).get({ id: ID }) as problemType;

            if (PROBLEM){
                  if (USER === PROBLEM.creator){
                        DB.prepare(`
                              DELETE FROM PROBLEMS
                              WHERE problem_id = (@id);
                        `).run({ id: ID });
                        return res.status(200).send();

                  } else {
                        return res.status(403).json({ error: "Not authorized for
the requested action" });
                  }
            }
      });

      ROUTER.all("/id", (req: Request, res: Response) => {
            res.set("Allow", "GET, DELETE");
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      return ROUTER;
}
```

src/routes/exception.route.ts

```
import { Request, Response, Router } from "express";

export function elseRouter(){
      const ROUTER = Router();

      // Triggers if content that doesn't exist (404) is requested,
      // also works if the user uses a URL path ID in a way that's
      // unsupported.
      ROUTER.use((req: Request, res: Response, next: Function) => {
            return res.status(404).json({ error: 'Not Found' });
      });

      return ROUTER;
}
```

src/routes/index.route.ts

```
import { Request, Response, Router } from "express";
```

```
export function indexRouter(){
	const ROUTER = Router();

	ROUTER.get("/", (req: Request, res: Response) => {
		if (!req.session.user){
			res.render("index", { login_status: "Not logged in", login:
false} );
			return;
		}
		if (req.session.user){
			let loginStatus = req.session.user;
			res.render("index", { login_status: loginStatus, login: true }
);
			return;
		}
	});

	ROUTER.all("/", (req: Request, res: Response) => {
		res.set("Allow", "GET");
		res.status(405).json({ error: "HTTP method not allowed" });
	});

	return ROUTER;
}
```

src/routes/leaderboard.route.ts

```
import { Request, Response, Router } from 'express'
import sqlite3 from "better-sqlite3";

export function leaderboardRouter(db: sqlite3.Database){
	const ROUTER = Router();

	ROUTER.get("/", async (req: Request, res: Response) => {
		const ATTEMPTS = db.prepare(`
			SELECT * FROM Attempts
			INNER JOIN Problems ON Attempts.problem_id =
Problems.problem_id
			ORDER BY elo DESC
			LIMIT 50;
		`);

		const ATTEMPTS_RESULT = ATTEMPTS.all();
```

```
            res.render("leaderboard", { attempts: ATTEMPTS_RESULT });
      });

      ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "GET");
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      return ROUTER;
}
```

src/routes/login.route.ts

```
import { Request, Response, Router } from 'express'
import bcrypt from "bcrypt";
import sqlite3 from "better-sqlite3";

export function loginRouter(DB: sqlite3.Database){
      const ROUTER = Router();

      interface userPassword {
            password: string;
      }

      ROUTER.get("/", (req: Request, res: Response) => {
            res.render("login");
      });

      ROUTER.post("/", async (req: Request, res: Response) => {
            const USER_INPUT = req.body;

            if (!USER_INPUT || !USER_INPUT.username || !USER_INPUT.password) {
                  return res.status(412).json({ error: "Required credentials are
missing" });
            }

            const RESULT = DB.prepare(`
                  SELECT password FROM Users
                  WHERE username = ?
            `).get(USER_INPUT.username) as userPassword | undefined;

            if (!RESULT) {
                  return res.status(404).json({ error: "User doesn't exist" });
            }
```

```
            const INPUT_PASS: string = USER_INPUT.password;
            const HASHED_PASS: string = RESULT.password;

            async function verifyPassword(INPUT_PASS: string, HASHED_PASS:
string): Promise<boolean> {

                  const MATCH: boolean = await bcrypt.
                  compare(INPUT_PASS, HASHED_PASS);

                  return MATCH;
            }

            const MATCH: boolean = await verifyPassword(INPUT_PASS,
HASHED_PASS);

            if (MATCH) {
                  req.session.user = USER_INPUT.username;
                  return res.status(200).send();

            } else {
                  return res.status(401).json({ error: "Incorrect password or
username" });
            }
      });


      ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "GET, POST");
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      return ROUTER;
}
```

src/routes/logout.route.ts

```
import { Request, Response, Router } from 'express'

export function logoutRouter(){
      const ROUTER = Router();

      // This could be a DELETE request
      ROUTER.post("/", (req: Request, res: Response) => {
            req.session.destroy((err) => {
                  if (err) {
                        console.log("Error destroying session", err);
```

```
                        return res.status(500).json({
                              error: "Error destroying session"
                        });
                  }

                  res.clearCookie("connect.sid");
                  // cookiePerm = false; // leftover from /disable-cookies route
                  return res.status(200).send();
            }
      )});

      ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "POST");
            res.status(405).json({
                  error: "HTTP method not allowed"
            });
      });

      return ROUTER;
}
```

src/routes/redirect.route.ts

```
import { Request, Response, Router } from "express";
import sqlite3 from "better-sqlite3";

export function redirectRouter(db: sqlite3.Database){
      const ROUTER = Router();

      interface redirectTypes{
            title: string
            problem_id: number
      }

      ROUTER.get("/:id", (req: Request, res: Response) => {
            const ID = req.params.id;
            res.render("create_success", { id: ID });
      });

      ROUTER.search("/", (req: Request, res: Response) => {
            const ID = req.body as redirectTypes;

            const USER_SEARCH = db.prepare(`
                  SELECT problem_id
                  FROM Problems WHERE title = ?
```

```
            `).get(ID.title) as redirectTypes;

            if (USER_SEARCH){
                  const URL = "/redirect/" + USER_SEARCH.problem_id;
                  return res.redirect(303, URL);
            } else {
                  return res.status(404).json({ error: "Couldn't find problem_id
using title" });
            }
      });


      ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "SEARCH");
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      ROUTER.all("/:id", (req: Request, res: Response) => {
            res.set("Allow", "GET");
            res.status(501).json({ error: "Using a URL path ID isn't
supported"});
}     );

      return ROUTER;
}
```

src/routes/search.route.ts

```
import { Request, Response, Router } from 'express'
import sqlite3 from "better-sqlite3";

export function searchRouter(db: sqlite3.Database){
      const ROUTER = Router();

      ROUTER.get("/", async (req: Request, res: Response) => {
            const QUERY_ELO_UPPER = req.query.fu as string; // "fu": filter
upper
            const QUERY_ELO_LOWER = req.query.fl as string; // ...: ... lower
l           et conditions = assignFilters(QUERY_ELO_UPPER, QUERY_ELO_LOWER) as
string;

            const QUERY_ORDER = req.query.order as string;
            let order = assignOrder(QUERY_ORDER) as string;

            const QUERY_SORT = req.query.sort as string;
            let sort = assignSort(QUERY_SORT, order) as string;
```

```
            const QUERY_PAGE = req.query.page as string;
            const LIMIT = 50;
            const OFFSET = assignOffset(QUERY_PAGE, LIMIT);

            if (conditions === null || order === null || sort === null || OFFSET
=== null){
                  return res.status(412).json({ error: "Invalid data supplied
with a valid query param" });
            }

            let query: string = "";
            let innerJoin = "";
            let columns: string = "problem_id, title, creator, time_created,
elo, times_attempted";
            let activity: string = "Searching";
            const UNSANITIZED_QUERY = req.query.q as string;

            [
                  query,
                  innerJoin,
                  columns,
                  conditions,
                  sort,
                  activity
            ] = assignQuery(UNSANITIZED_QUERY, query, innerJoin, columns,
conditions, sort, activity);

            let results: sqlite3.RunResult | unknown[] = db.prepare(`
                  SELECT ${columns}
                  FROM Problems
                  ${innerJoin}
                  WHERE ${conditions}
                  ORDER BY ${sort}
                  LIMIT (@limit) OFFSET (@offset);
            `).all({
                  query: query,
                  limit: LIMIT,
                  offset: OFFSET
            });

            res.render("search", {
                  activity: activity,
                  results: results,
                  userSearch: req.query.q,
            });
```

```
	});

	function assignFilters(QUERY_ELO_UPPER: string, QUERY_ELO_LOWER: string){
		let eloUpper: number;
		let eloLower: number = 0;

		let conditions: string | null = "elo > " + eloLower; // + "AND elo <
(@upper)";

		if (QUERY_ELO_LOWER){
			const PARSED = Number(QUERY_ELO_LOWER);
			if (!isNaN(PARSED)){
				eloLower = PARSED;
				conditions = "elo > " + eloLower;
			} else {
				return conditions = null;
			}
		} if (QUERY_ELO_UPPER){
			const PARSED = Number(QUERY_ELO_UPPER);
			if (!isNaN(PARSED)){
				eloUpper = PARSED;
				conditions = "elo > " + eloLower + " AND elo < " +
eloUpper;
			} else {
				return conditions = null;
			}
		}

		return conditions;
	}

	function assignOrder(QUERY_ORDER: string){
		let order: string | null = "DESC";
		let orderWhitelist: string[];

		if (QUERY_ORDER){
			orderWhitelist = ["DESC", "ASC"];
			if (orderWhitelist.includes(QUERY_ORDER.toString())){
				order = QUERY_ORDER.toString();
			} else {
				return order = null;
			}
		}

		return order;
	}
```

```
	function assignSort(QUERY_SORT: string, order: string){
		let sort: string = "time_created";
		let sortWhitelist: string[];

		if (QUERY_SORT){
			sortWhitelist = ["elo", "time_created", "times_attempted"];
			if (sortWhitelist.includes(QUERY_SORT.toString())){
				sort = QUERY_SORT.toString();
			} else {
				return null;
			}
		}

		return (sort + " " + order);
	}

	function assignOffset(QUERY_PAGE: string, LIMIT: number, page = 1){
		if (QUERY_PAGE){
			const PARSED = Number(QUERY_PAGE);
			if (!isNaN(PARSED)){
				page = PARSED;
			} else {
				return null;
			}
		}

		return ((page - 1) * LIMIT);
	}

	function assignQuery(UNSANITIZED_QUERY: string, query: string, innerJoin: string,
columns: string, conditions: string, sort: string, activity: string){

		if (UNSANITIZED_QUERY){
			query = UNSANITIZED_QUERY.toString();
			const SYMBOL_STRIP: RegExp = /[^a-zA-Z0-9 ]/g;
			query = query.replace(SYMBOL_STRIP, "").trim().slice(0, 100);
			query = query + "*";

			innerJoin = "INNER JOIN Problems_fts ON Problems.problem_id = Problems_fts.rowid";
			columns = "Problems.*";
			conditions = "Problems_fts MATCH (@query)";
			sort = "rank";
```

```
                activity = "after_query";
            }

            return [query, innerJoin, columns, conditions, sort, activity];
        }

        ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "GET");
            res.status(405).json({ error: "HTTP method not allowed" });
        });

        return ROUTER;
}
```

src/routes/signup.route.ts

```
import { Request, Response, Router } from "express";
import sqlite3 from "better-sqlite3";
import bcrypt from "bcrypt";

export function signupRouter(DB: sqlite3.Database){
        const ROUTER = Router();

        ROUTER.get("/", (req: Request, res: Response) => {
            res.render("signup");
        });

        ROUTER.post("/", async (req: Request, res: Response) => {
            const USER_INPUT = req.body;

            if (!USER_INPUT || !USER_INPUT.username || !USER_INPUT.password) {
                return res.status(412).json({ error: "Required fields are
missing" });
            }

            const existingUser = DB.prepare(`
                SELECT username FROM Users
                WHERE username = ?
            `).get(USER_INPUT.username);

            if (existingUser) {
                return res.status(409).json({ error: "Username already exists"
});
            }

            const ERROR = testInput(USER_INPUT);
```

```
            if (ERROR === true){
                  return res.status(412).send();
            }
            async function hashPassword(PASS: string): Promise<string> {
                  const SALT_ROUNDS = 10;
                  const HASHED_PASS = await bcrypt.hash(PASS, SALT_ROUNDS);
                  return HASHED_PASS;
            }

            const HASHED_PASS = await hashPassword(USER_INPUT.password);

            const INSERT = DB.prepare(`
                  INSERT INTO Users (username, password)
                  VALUES (@username , @password);
            `);

            try {
                  INSERT.run({ username: USER_INPUT.username, password:
HASHED_PASS });

            } catch (err) {
                  console.log(err);
                  return res.status(500).json({ error: err });
            }

            return res.status(200).send();
      });

      interface inputTypes {
            username: string,
            password: string,
      }

      function testInput(USER_INPUT: inputTypes){
            if ((USER_INPUT.password).length < 6) {
                  return true;
            }

            if ((USER_INPUT.password).length < 6) {
                  return true;
            }

            if ((USER_INPUT.username).length > 16) {
                  return true;
            }
```

```
            let num = false;
            let symbol = false;
            let lower = false;
            let upper = false;

            for (let i = 0; i < USER_INPUT.password.length; i++){
                  const char: string = USER_INPUT.password[i];

                  if (char ≥ "0" && char ≤ "9"){
                        num = true;
                  } if (!/[a-zA-Z0-9]/.test(char)){
                        symbol = true;
                  } if (/[a-z]/.test(char)){
                        lower = true;
                  } if (/[A-Z]/.test(char)){
                        upper = true;
                  }
            }

            if (num === false || symbol === false || lower === false || upper
=== false){
                  return true;
            }

            return false;

      }

      ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "GET, POST");
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      return ROUTER;
}
```

src/routes/solution.route.ts

```
import { Request, Response, Router } from "express";
import sqlite3 from "better-sqlite3";

export function solutionRouter(DB: sqlite3.Database){
      const ROUTER = Router();

      interface types {
```

```
            test_data: string,
            answer: string,
      }

      ROUTER.get("/:id", async(req: Request, res: Response) => {
            const ID = req.params.id;

            try {
                  const response = DB.prepare(`SELECT answer, test_data FROM
Problems WHERE problem_id = ?;`).get(ID);

                  // Useful for testing purposes
                  console.log("answer = " + (response as types).answer);

                  return res.json(response);

            } catch(error){
                  return res.status(500).json({ error: "Unknown server error"
});
            }
      });

      ROUTER.all("/:id", (req: Request, res: Response) => {
            res.set("Allow", "GET");
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      return ROUTER;
}
```

src/routes/solve.route.ts

```
import { Request, Response, Router } from "express";
import sqlite3 from "better-sqlite3";

export function solveRouter(DB: sqlite3.Database){
      const ROUTER = Router();

      interface fetch {
            id: string;
            creator: string,
      }

      ROUTER.get("/:id", async(req: Request, res: Response) => {
            const ID = req.params.id;
```

```
            const info = DB.prepare(`
                  SELECT * FROM Problems
                  WHERE problem_id = ?;
            `).get(ID) as fetch | undefined;

            const attempts = DB.prepare(`
                  SELECT Attempts.*, Users.elo
                  FROM Attempts
                  INNER JOIN Problems ON Attempts.problem_id =
Problems.problem_id
                  INNER JOIN Users ON Attempts.username = Users.username
                  WHERE Attempts.problem_id = ?
                  LIMIT 10;
            `).all(ID);

            if (info){
                  if (req.session.user === info.creator){
                        res.render("solve", {
                              info: info,
                              attempts: attempts,
                              auth: true,
                        });
                  } else {
                        res.render("solve", {
                              info: info,
                              attempts: attempts,
                              auth: false,
                        });
                  }
            }
      });

      ROUTER.all("/:id", (req: Request, res: Response) => {
            res.set("Allow", "GET");
            res.status(405).json({ error: "HTTP method not allowed" });

      });

      return ROUTER;
}
```

src/routes/submit_attempt.route.ts

```
import { Request, Response, Router } from "express";
import sqlite3 from "better-sqlite3";
```

```
export function attemptRouter(db: sqlite3.Database){
	const ROUTER = Router();

	interface payloadTypes {
		correct: boolean,
		problemId: string,
	}

	interface dbTypes {
		elo: number,
		tries: number,
		solved: number,
	}

	interface checkTypes{
		count: number,
	}

	class Elo {
		private kFactor: number;
		public outcome: number;

		private userProbability: number = 0;
		private problemProbability: number = 0;

		private userElo: number = 0;
		private problemElo: number = 0;
		public newUserElo: number = 0;
		public newProblemElo: number = 0;

		constructor(kFactor: number, outcome: number) {
			this.kFactor = kFactor;
			this.outcome = outcome;
		}

		fetchElo(user: string | undefined, attempt: payloadTypes){
			let FETCH_USER = db.prepare(`
				SELECT elo FROM Users
				WHERE username = ?
			`).get(user) as dbTypes;

			let fetchProblem = db.prepare(`
				SELECT elo FROM Problems
				WHERE problem_id = ?`
			).get(attempt.problemId) as dbTypes;
			this.userElo = FETCH_USER.elo;
```

```
                this.problemElo = fetchProblem.elo;
            }

            probability(rating1: number, rating2: number){
                return 1 / (1 + Math.pow(10, (rating1 - rating2) / 400));
            }

            update(){
                this.userProbability = this.probability(this.problemElo,
this.userElo);
                this.problemProbability = this.probability(this.userElo,
this.problemElo);

                this.newUserElo = this.userElo + this.kFactor * (this.outcome
- this.userProbability);
                this.newProblemElo = this.problemElo + this.kFactor * (1 -
                this.outcome - this.problemProbability);

                // float -> int
                this.newUserElo = Math.trunc(this.newUserElo);
                this.newProblemElo = Math.trunc(this.newProblemElo);

                if (this.newUserElo < 100){
                    this.newUserElo = 100;
                }

                if (this.newProblemElo < 100){
                    this.newProblemElo = 100;
                }

                if (Number.isNaN(this.newUserElo) ||
Number.isNaN(this.newProblemElo)){
                    return Error;
                }
            }
        }

        class Attempts{
            private user: string | undefined;
            private attempt: payloadTypes;

            constructor(user: string | undefined, attempt: payloadTypes){
                this.user = user;
                this.attempt = attempt;
            }
```

```
        check(){
            const EXISTS = db.prepare(`
                SELECT COUNT(1)
                AS count
                FROM Problems
                WHERE problem_id = (@id);
            `).get({id: this.attempt.problemId}) as checkTypes;

            if (EXISTS.count === 1){
                return false;
            } else {
                return true;
            }
        }

        increment(attempt: payloadTypes){
            db.prepare(`
                UPDATE problems
                SET times_attempted = times_attempted + 1
                WHERE problem_id = (@id);
            `).run({id: attempt.problemId});

            if (attempt.correct === true) {
                db.prepare(`
                    UPDATE Problems
                    SET times_solved = times_solved + 1
                    WHERE problem_id = (@id);
                `).run({id: attempt.problemId});
            }
        }

        record(E: Elo){
            // Redirects user based on whether they already have
            // history on a problem, then calls method to update
            // elo.
            const FETCH_USER = db.prepare(`
                SELECT solved FROM Attempts
                WHERE username = (@user) AND
                problem_id = (@id);
            `).get({ user: this.user, id: this.attempt.problemId }) as
dbTypes;

            if (!FETCH_USER){
                this.attemptInsert(E);
                this.eloUpdate(E);
            } else if (FETCH_USER.solved === 0){
```

```
                        this.attemptUpdate(E);
                        this.eloUpdate(E);
                  } else if (FETCH_USER.solved === 1){
                        // If a problem has been solved, then the user shouldn't
                        // gain more elo from it, it would defeat the purpose
since
/                       / people could farm easy problems.
                  } else {
                        this.attemptInsert(E);
                        this.eloUpdate(E);
                  }
            }

            attemptInsert(E: Elo){
                  const NEW_ATTEMPT = db.prepare(`
                        INSERT INTO Attempts(username, problem_id, solved)
                        VALUES(@username, @problem_id, @solved);
                  `).run({
                        username: this.user,
                        problem_id: this.attempt.problemId,
                        solved: E.outcome
                  });
            }

            attemptUpdate(E: Elo){
                  db.prepare(`
                        UPDATE Attempts
                        SET tries = tries + 1,
                        solved = (@solved)
                        WHERE username = (@user) AND
                        problem_id = (@id);
                  `).run({solved: E.outcome, user: this.user, id:
this.attempt.problemId});
            }

            eloUpdate(E: Elo){
                  db.prepare(`
                        UPDATE Users
                        SET elo = (@userElo)
                        WHERE username = (@user);
                  `).run({userElo: E.newUserElo, user: this.user});

                  db.prepare(`
                        UPDATE Problems
                        SET elo = (@elo)
                        WHERE problem_id = (@id);
```

```
                `).run({elo: E.newProblemElo, id: this.attempt.problemId});
            }

        }
        ROUTER.post("/", async (req: Request, res: Response) => {
            try {
                let user = req.session.user as string | undefined;
                let attempt: payloadTypes = req.body;

                let A = new Attempts(user, attempt);

                const ERROR: boolean = A.check();
                if (ERROR === true){
                    return res.status(404).json({ error: "Problem doesn't
exist?" });
                }

                A.increment(attempt);

                if (user){
                    // 1 = user winner, 0 = user loser
                    let outcome: number;

                    if (attempt.correct === true){ outcome = 1; }
                    else outcome = 0;

                    const E = new Elo(25, outcome);
                    E.fetchElo(user, attempt);
                    E.update();

                    A.record(E);
                }

                return res.status(200).send();

            } catch (Error) {
                console.log(Error);
                return res.status(500).json({ error: "Server side error" });
            }
        });

        ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "POST");
            res.status(405).json({ error: "HTTP method not alllowed" });
        });
```

```
	return ROUTER;
}
```

src/routes/user.route.ts

```
import { Request, Response, Router } from "express";
import sqlite3 from "better-sqlite3";

export function userRouter(DB: sqlite3.Database){
	const ROUTER = Router();

	class Get {
		public TARGET;

		constructor(ID: string | undefined, USER: string | undefined){
			if (ID){
				this.TARGET = ID;
			} else if (USER){
				this.TARGET = USER;
			}
		}

		userInfo(){
			return DB.prepare(`
				SELECT username, elo, time_created
				FROM Users WHERE username = (@username);
			`).get({ username: this.TARGET });
		}

		attempts(){
			return DB.prepare(`
				SELECT * FROM Attempts
				INNER JOIN Problems ON Attempts.problem_id =
Problems.problem_id
				WHERE Attempts.username = (@username)
				AND Attempts.solved = 1
				ORDER BY Problems.elo DESC
				LIMIT 5;
			`).all({ username: this.TARGET });
		}

		averageEloSolved(){
			return DB.prepare(`
				SELECT AVG(Problems.elo)
				AS avg_elo
				FROM Attempts
```

```
                        INNER JOIN Problems ON Attempts.problem_id =
Problems.problem_id
                        WHERE Attempts.username = (@username);
                  `).get({ username: this.TARGET });
            }

            problems(){
                  return DB.prepare(`
                        SELECT * FROM Problems
                        WHERE creator = (@username)
                        ORDER BY time_created DESC;
                  `).all({ username: this.TARGET });
            }
      }

      ROUTER.get("/:id", (req: Request, res: Response) => {
            const ID = req.params.id;
            const USER = req.session.user;

            const G = new Get(ID, USER);
            const userInfo = G.userInfo();

            if (G.TARGET && userInfo){
                  const averageEloSolved = G.averageEloSolved();
                  const attempts = G.attempts();
                  const problems = G.problems();

                  if (req.session.user === G.TARGET){
                        return res.render("user", {
                              results: userInfo,
                              average: averageEloSolved,
                              attempts: attempts,
                              problems: problems,
                              auth: true,
                        });
                  }

                  return res.render("user", {
                        results: userInfo,
                        average: averageEloSolved,
                        attempts: attempts,
                        problems: problems,
                        auth: false,
                  });
            }
```

```
            return res.render("user", {
                  results: null,
                  average: null,
                  attempts: null,
                  problems: null,
                  auth: false,
            });
      });

      ROUTER.get("/", (req: Request, res: Response) => {
            const ID = undefined;
            const USER = req.session.user;
            const G = new Get(ID, USER);

            const userInfo = G.userInfo();

            if (G.TARGET && userInfo){
                  const averageEloSolved = G.averageEloSolved();
                  const attempts = G.attempts();
                  const problems = G.problems();

                  return res.render("user", {
                        results: userInfo,
                        average: averageEloSolved,
                        attempts: attempts,
                        problems: problems,
                        auth: true,
                  });
            }

            return res.render("user", {
                  results: null,
                  average: null,
                  attempts: null,
                  problems: null,
                  auth: false,
            });
      });

      ROUTER.all("/", (req: Request, res: Response) => {
            res.set("Allow", "GET");
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      ROUTER.all("/:id", (req: Request, res: Response) => {
            res.set("Allow", "GET");
```

```
            res.status(405).json({ error: "HTTP method not allowed" });
      });

      return ROUTER;
}
```

src/client/create.ts

```
const PROBLEM_FORM = document.
getElementById("problem_form") as HTMLFormElement;
const ERROR_DISPLAY = document.
getElementById("errP") as HTMLParagraphElement;

if (PROBLEM_FORM){
      PROBLEM_FORM.addEventListener("submit", (event) => {
event.preventDefault();
            const TITLE = (document.getElementById("title") as
HTMLInputElement).value;
            const INSTRUCTION = (document.getElementById("instr") as
HTMLInputElement).value;
            const TEST_CASE = (document.getElementById("exmpl") as
HTMLInputElement).value;
            const ANSWER_STRING = (document.getElementById("answ") as
HTMLInputElement).value;

            const DELIM_TEST_REGEX: RegExp = /\/(\w+)/ig;
            const DELIMS_PRESENT = DELIM_TEST_REGEX.exec(ANSWER_STRING);

            if (TITLE.trim() === "" || INSTRUCTION.trim() === "" ||
ANSWER_STRING.trim() === "" || TEST_CASE.trim() === "") {
                  ERROR_DISPLAY.innerHTML = "Don't leave empty fields!";

            } else if (DELIMS_PRESENT){
                  ERROR_DISPLAY.innerHTML = "Don't include delimiters in your
regular expression.";

            } else {
                  const ANSWER_REGEX = new RegExp(ANSWER_STRING);
                  ERROR_DISPLAY.innerHTML = "";
                  regexTest(TITLE, INSTRUCTION, ANSWER_STRING, ANSWER_REGEX,
TEST_CASE);
            }
      });
}
```

```
function regexTest(TITLE: string, INSTRUCTION: string, ANSWER_STRING: string,
ANSWER_REGEX: RegExp, TEST_CASE: string){
      if (ANSWER_REGEX.test(TEST_CASE)){
            createConfirm(TITLE, INSTRUCTION, ANSWER_STRING, ANSWER_REGEX,
TEST_CASE);
      } else {
            return ERROR_DISPLAY.innerHTML = "Please use JS regex, and include
delimiters!";
      }
}

function createConfirm(TITLE: string, INSTRUCTION: string, ANSWER_STRING:
string, ANSWER_REGEX: RegExp, TEST_CASE: string){
      document.getElementById("problem_form")!.remove();
      document.getElementById("cheatsheet")!.remove();

      const ANSWER_REGEX_RESULT = ANSWER_REGEX.exec(TEST_CASE);

      if (ANSWER_REGEX_RESULT === null){
            window.location.assign("http://localhost:8000/create");
      return ERROR_DISPLAY.innerHTML = ("Regex expression didn't match anything,
please try again!");

      }

      const confirmMsg = document.getElementById("confirm_msg");
      const confirmMsgChild = document.
createTextNode("Is [" + ANSWER_REGEX_RESULT + "] the data out of [" + TEST_CASE
+ "] that you want to use?");
      confirmMsg!.appendChild(confirmMsgChild);
      const aButtonContainer = document.getElementById("a_button_container");
      let agreeButton = document.createElement("BUTTON");
      let aButtonNode = document.createTextNode("Yep");

      agreeButton.appendChild(aButtonNode);
      agreeButton.id = ("submit_button");
      agreeButton.className = ("btn btn-light btn-lg");
      aButtonContainer!.appendChild(agreeButton);

      const dButtonContainer = document.getElementById("b_button_container");
      let disagreeButton = document.createElement("BUTTON");
      let dButtonNode = document.createTextNode("Cancel creation");

      disagreeButton.appendChild(dButtonNode);
      disagreeButton.id = ("submit_button");
      disagreeButton.className = ("btn btn-light btn-lg");
```

```
	dButtonContainer!.appendChild(disagreeButton);

	agreeButton.onclick = function() {
		return problemPost(TITLE, INSTRUCTION, ANSWER_STRING, TEST_CASE);
	}

	disagreeButton.onclick = function() {
		return window.location.href = "http://localhost:8000/create";
	}
}

async function problemPost(TITLE: string, INSTRUCTION: string, ANSWER_STRING:
string, TEST_CASE: string) {
	const RESPONSE = await fetch("http://localhost:8000/create", {
		method: "POST",
		headers: {
			"Content-Type": "application/json",
		},
		body: JSON.stringify({
			title: TITLE,
			instruction: INSTRUCTION,
			answer: ANSWER_STRING,
			test_data: TEST_CASE
		}),
	});

	return afterPost(RESPONSE, TITLE);
}

function afterPost(RESPONSE: Response, TITLE: string){
	if (RESPONSE.status === 200){
		return redirectSetup(TITLE);
	} if (RESPONSE.status === 204){
		return ERROR_DISPLAY.innerHTML = ("Please login first!");
	} else {
		return console.log(RESPONSE.status);
	}
}

async function redirectSetup(TITLE: string) {
	const RESPONSE = await fetch("http://localhost:8000/redirect", {
		method: "SEARCH",
		headers: {
			"Content-Type": "application/json",
		},
		body: JSON.stringify({
```

```
                title: TITLE
            },
        });
        return redirect(RESPONSE);
}

function redirect(RESPONSE: Response){
        if (RESPONSE.status === 200){
            return window.location.assign(RESPONSE.url);
        }
}
```

src/client/solve.ts

```
interface solutionTypes {
        test_data: string,
        answer: string,
}

async function getSolution(problemId: string | null) {
        const FETCH_URL = "http://localhost:8000/solution/" + problemId;
        const RESPONSE: Response = await fetch(FETCH_URL, {
            method: "GET",
            headers: {
                "Content-Type": "application/json",
            },
        });

        if (RESPONSE.status === 200){
            return await RESPONSE.json() as solutionTypes;
        } else {
            return window.location.assign("http://localhost:8000/search")
        }
}

async function submitAttempt(problemId: string | null, CORRECT: boolean) {
        const FETCH_URL = "http://localhost:8000/submit_attempt";
        const RESPONSE: Response = await fetch(FETCH_URL, {
            method: "POST",
            headers: {
                "Content-Type": "application/json",
            },
            body: JSON.stringify({
                correct: CORRECT,
                problemId: problemId,
            }),
```

```
	});

	if (RESPONSE.status === 200){
		return;
	} else {
		return console.log(RESPONSE.status);
	}
}

class Data {
	private problemId: string | null;
	public problemData: solutionTypes;

	async fetchData(PAGE_URL: string){
		this.problemId = PAGE_URL.match(/\d+$/)?.[0] ?? null;
		if (!this.problemId) throw new Error("problemId is empty?");
		this.problemData = await getSolution(this.problemId) as
solutionTypes;
	}

	async submitAttempt(PAGE_URL: string, CORRECT: boolean){
		this.problemId = PAGE_URL.match(/\d+$/)?.[0] ?? null;
		if (!this.problemId) throw new Error("problemId is empty?");
		await submitAttempt(this.problemId, CORRECT);
	}
}

// Allows the code to be executed before the frontend is completely loaded:
(async () => {
	const PAGE_URL = window.location.href;
	const D = new Data();
	await D.fetchData(PAGE_URL);

	if (D.fetchData =/= null && D.fetchData =/= undefined){
		form(PAGE_URL, D);
	}

})();

async function form(PAGE_URL: string, D: Data){
	const SOLUTION_FORM = document.
	getElementById("solution_form") as HTMLFormElement;

	SOLUTION_FORM.addEventListener("submit", (event) => {
		event.preventDefault();
```

```
            const INPUT_SOLUTION: string = (document.getElementById("solution")
as HTMLInputElement).value;

            if (INPUT_SOLUTION =/= null){
                  if (INPUT_SOLUTION === D.problemData.answer) {
                        postSolve(PAGE_URL, true, D, INPUT_SOLUTION);
                  } else {
                        postSolve(PAGE_URL, false, D, INPUT_SOLUTION);
                  }
            }
      });
}

async function postSolve(PAGE_URL: string, CORRECT: boolean, D: Data,
INPUT_SOLUTION: string){
      clearScreen();
      await D.submitAttempt(PAGE_URL, CORRECT);
      resultsDisplay(CORRECT, D, INPUT_SOLUTION);
      redirectButtonsSetup(PAGE_URL);
}

function clearScreen(){
      document.getElementById("pre_solution")!.remove();
      document.getElementById("recent_attempts")!.remove();
      document.getElementById("solution_form")!.remove();
}

function resultsDisplay(CORRECT: boolean, D: Data, INPUT_SOLUTION: string){
      const JUDGEMENT = document.
      getElementById("right_or_wrong");
      let judgeChild: Text;

      if (CORRECT === true){
            judgeChild = document.createTextNode("Correct!");
      } else {
            judgeChild = document.createTextNode("Incorrect");
      }

      JUDGEMENT!.appendChild(judgeChild);

      // Database data
      const TEST_CASE_DISPLAY = document.
      getElementById("test_data");
      const TEST_DISPLAY_CHILD = document.
      createTextNode("Test case: [" + D.problemData.test_data + "]");
      TEST_CASE_DISPLAY!.appendChild(TEST_DISPLAY_CHILD);
```

```
	const SOLUTION_DISPLAY = document.
	getElementById("db_solution");
	const SOLUTION_REGEX = new RegExp(D.problemData.answer);
	const SOLUTION_RESULT: RegExpExecArray | null = SOLUTION_REGEX.
	exec(D.problemData.test_data);
	const SOLUTION_DISPLAY_CHILD = document.
	createTextNode("Intended result: [" + SOLUTION_RESULT + "]");
	SOLUTION_DISPLAY!.appendChild(SOLUTION_DISPLAY_CHILD);

	// User data
	const ATTEMPT_DISPLAY = document.
	getElementById("user_attempt");
	const ATTEMPT_REGEX = new RegExp(INPUT_SOLUTION);
	const ATTEMPT_RESULT: RegExpExecArray | null = ATTEMPT_REGEX.
	exec(D.problemData.test_data);
	const ATTEMPT_DISPLAY_CHILD = document.
	createTextNode("Your result: [" + ATTEMPT_RESULT + "]");
	ATTEMPT_DISPLAY!.appendChild(ATTEMPT_DISPLAY_CHILD);
}

function redirectButtonsSetup(PAGE_URL: string){
	const HOME_BUTTON_CONTAINER =
document.getElementById("h_button_container");
	const HOME_BUTTON = document.createElement("BUTTON");
	const HOME_BUTTON_CHILD = document.createTextNode("Go home");

	HOME_BUTTON.appendChild(HOME_BUTTON_CHILD);
	HOME_BUTTON.id = ("submit_button");
	HOME_BUTTON.className = ("btn btn-light btn-lg"); // Appending bootstrap
classes
	HOME_BUTTON_CONTAINER!.appendChild(HOME_BUTTON);

	const RETRY_BUTTON_CONTAINER =
document.getElementById("r_button_container");
	const RETRY_BUTTON = document.createElement("BUTTON");
	const RETRY_BUTTON_CHILD = document.createTextNode("Retry!");

	RETRY_BUTTON.appendChild(RETRY_BUTTON_CHILD);
	RETRY_BUTTON.id = ("submit_button");
	RETRY_BUTTON.className = ("btn btn-light btn-lg");
	RETRY_BUTTON_CONTAINER!.appendChild(RETRY_BUTTON);

	HOME_BUTTON.onclick = function() {
		return window.location.assign("/");
	}
```

```
	RETRY_BUTTON.onclick = function() {
		return window.location.assign(PAGE_URL);
	}
}
```